

Algoritmos de travessia de grafos

Os algoritmos de travessia de grafos são utilizados para visitar todos os vértices de um grafo de forma organizada.

Algoritmos de travessia de grafos

Os dois algoritmos mais conhecidos são:

Busca em Largura (BFS - Breadth First Search)

Busca em Profundidade (DFS - Depth First Search)

Uma analogia simples:

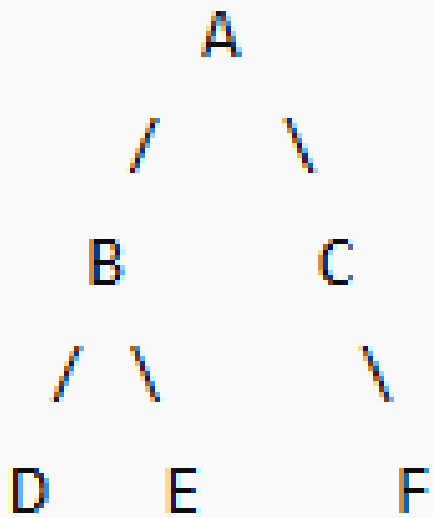
BFS → visita primeiro os vizinhos mais próximos.

DFS → segue um caminho até o fim antes de voltar.

Algoritmos de travessia de grafos

Grafo de Exemplo

Vamos utilizar o seguinte grafo:



Algoritmos de travessia de grafos

Representando os vértices por números:

A = 0

B = 1

C = 2

D = 3

E = 4

F = 5

Lista de adjacência:

0 -> 1 -> 2

1 -> 3 -> 4

2 -> 5

3

4

5

Algoritmos de travessia de grafos

1. Busca em Largura (BFS)

A BFS visita os vértices por níveis.

Imagine uma pedra caindo em um lago:

Nível 0: A

Nível 1: B C

Nível 2: D E F

A ordem de visita será:

A → B → C → D → E → F

Algoritmos de travessia de grafos

Como a BFS funciona

Ela utiliza uma **fila (Queue)**.

Passo 1

Fila:

[A]

Visita:

A

Algoritmos de travessia de grafos

Passo 2

Adiciona os vizinhos de A:

[B, C]

Visita:

$A \rightarrow B$

Algoritmos de travessia de grafos

Passo 3

Remove B e adiciona seus vizinhos:

[C, D, E]

Visita:

$A \rightarrow B \rightarrow C$

Algoritmos de travessia de grafos

Passo 4

Remove C e adiciona F:

[D, E, F]

Visita:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$

Algoritmos de travessia de grafos

```
#include <stdio.h>
#define V 6
void BFS(int grafo[V][V], int inicio) {
    int visitado[V] = {0};
    int fila[V];
    int inicioFila = 0;
    int fimFila = 0;
    visitado[inicio] = 1;
    fila[fimFila++] = inicio;
    while (inicioFila < fimFila) {
        int atual = fila[inicioFila++];
        printf("%d ", atual);
        for (int i = 0; i < V; i++) {
            if (grafo[atual][i] == 1 &&
                !visitado[i]) {
                visitado[i] = 1;
                fila[fimFila++] = i;
            }
        }
    }
}
```

Algoritmos de travessia de grafos

```
int main() {  
  
    int grafo[V][V] = {  
  
        //A B C D E F|  
        {0,1,1,0,0,0}, // A  
        {0,0,0,1,1,0}, // B  
        {0,0,0,0,0,1}, // C  
        {0,0,0,0,0,0}, // D  
        {0,0,0,0,0,0}, // E  
        {0,0,0,0,0,0}  // F  
    };  
  
    printf("BFS: ");  
    BFS(grafo,0);  
  
    return 0;  
}
```

Algoritmos de travessia de grafos

```
int main() {  
  
    int grafo[V][V] = {  
  
        //A B C D E F|  
        {0,1,1,0,0,0}, // A  
        {0,0,0,1,1,0}, // B  
        {0,0,0,0,0,1}, // C  
        {0,0,0,0,0,0}, // D  
        {0,0,0,0,0,0}, // E  
        {0,0,0,0,0,0}  // F  
    };  
  
    printf("BFS: ");  
    BFS(grafo,0);  
  
    return 0;  
}
```

Algoritmos de travessia de grafos

2. Busca em Profundidade (DFS)

A DFS tenta ir o mais fundo possível antes de voltar.

Pense em um labirinto:

Se existe um corredor,
continue andando.

Só volte quando encontrar
um beco sem saída.

Funcionamento

Começando em A:

A

↓

B

↓

D

Algoritmos de travessia de grafos

D não possui filhos.

Volta para B.

A

↓

B

↓

E

E termina.

Volta para A.

A

↓

C

↓

F

Ordem de visita

A → B → D → E → C → F

ou

0 → 1 → 3 → 4 → 2 → 5

Algoritmos de travessia de grafos

```
void DFS(int grafo[V][V],
        int vertice,
        int visitado[]) {

    visitado[vertice] = 1;

    printf("%d ", vertice);

    for(int i = 0; i < V; i++) {

        if(grafo[vertice][i] == 1 &&
           !visitado[i]) {

            DFS(grafo, i, visitado);

        }

    }

}
```

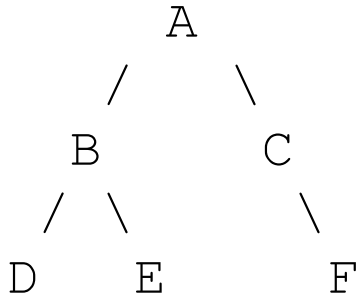
Algoritmos de travessia de grafos

```
int main() {  
  
    int grafo[V][V] = {  
  
        //A B C D E F  
        {0,1,1,0,0,0},  
        {0,0,0,1,1,0},  
        {0,0,0,0,0,1},  
        {0,0,0,0,0,0},  
        {0,0,0,0,0,0},  
        {0,0,0,0,0,0}  
    };  
  
    int visitado[V] = {0};  
  
    printf("DFS: ");  
  
    DFS(grafo,0,visitado);  
  
    return 0;  
}
```

Algoritmos de travessia de grafos

Visualizando a Diferença

Considere novamente:



BFS

Visita por níveis:

A

B C

D E F

Resultado:

A B C D E F

Algoritmos de travessia de grafos

DFS

Vai até o fundo primeiro:

A

↓

B

↓

D

(volta)

E

(volta)

C

↓

F

Resultado:

A B D E C F

Algoritmos de travessia de grafos

Estruturas Utilizadas

Algoritmo	Estrutura
BFS	Fila (Queue)
DFS	Pilha (Stack) ou Recursão

Algoritmos de travessia de grafos

Aplicações Práticas

BFS

- Encontrar o caminho mais curto em grafos sem peso.
- GPS simples.
- Redes sociais.
- Jogos.

Exemplo:

Qual o menor número de amizades entre João e Maria?

DFS

- Detectar ciclos.
- Resolver labirintos.
- Ordenação topológica.
- Encontrar componentes conexos.

Exemplo:

Existe algum caminho entre duas páginas da web?

Algoritmos de travessia de grafos

Resumo

Característica	BFS	DFS
Estratégia	Visita por níveis	Vai até o fundo
Estrutura	Fila	Pilha/Recursão
Ordem no exemplo	A B C D E F	A B D E C F
Menor caminho sem peso	Sim	Não
Uso comum	Rotas e distâncias	Exploração e análise

Regra fácil para decorar

- BFS (Busca em Largura)** → "Visita os irmãos primeiro".
- DFS (Busca em Profundidade)** → "Visita os filhos primeiro".

Essa é a principal diferença entre os dois algoritmos de travessia de grafos.